SpecChain Part 2 (Normative Construction): Q-Grade Algebra, Uncertainty Lift, and Frontier Accounting

SpecChain Working Draft

September 17, 2025

Abstract

Contract (what Part 2 exports). This part closes three obligations promised in Part 1: (i) it *instantiates* per-axis quantales and their typed product, yielding a complete, mechanizable acceptance algebra $\mathbf{Q}$ with *unknown as absorbing bottom* $\perp$ and *distributivity-on-demand* on declared complete axes; (ii) it *lifts* $\mathbf{Q}$ to an uncertainty-aware $\mathbf{Q}'$ via sound, order-theoretic carriers (intervals, sub-Gaussian envelopes, robust sets) and exports acceptance stances with guard lifting; (iii) it *defines and analyzes* the canonical frontier debit Δ_{fr}, proves the frontier axioms, establishes *strong NP-hardness* of exact evaluation, and specifies *sound upper-bounding* algorithms with verifiable receipts. These constructions are policy-driven and match the axis catalog/typed product surface from Part 1 (§10–§11); they align with the Loader's guard-first, fail-closed semantics and receipt discipline (Parts 3–4).

Notation hygiene. Internally we use $q \uplus r$ for the heterogeneous ("horizontal") aggregator (max on M, $+$ on S) and $q \star r$ for the sequential aggregator ($+$ on both). For cross-part consistency we expose aliases $\uplus \equiv \uplus$ (Part 5's $\odot$) and $\star \equiv \star$ (Part 1's product aggregator $\wedge$). Unknown/missing evidence is uniformly the lattice bottom $\perp$, *absorbing* for both aggregators and guarded upstream to prevent smuggling into priced contexts.

Scope. Formal statements include: product-quantale construction per axis policy, residuation (budget-as-implication) on complete slices, Galois-style sound lift to $\mathbf{Q}'$, frontier axioms with guard placement, strong NP-hardness (via 3-Partition), and soundness/tightness of greedy, bounded-width DP, and convex/robust relax-and-round (with monotone rounding receipts). Mechanization hooks (Lean-style) and minimal JSON schemas ensure byte-replayable audits and interoperation with the Loader and pricing kernels.

Contents

1 Preliminaries and Notation **3**

2 Operations on Q **4**

3 Quantale Upgrade (per-axis completeness) **5**

4 Uncertainty-aware Lift $\mathbf{Q}'$ **6**

4.1 Carriers and orders (with absorbing unknown) 6
4.2 Abstract interpretation interface 7
4.3 Typed operators on $\mathbf{Q}'$ 8
4.4 Acceptance stances and upward closure 8
4.5 Worked example: sub-Gaussian acceptance 8
4.6 Interoperability and receipts 8

5 Frontier Debit Δ_{fr}: Canonical Construction **9**
5.1 Admissible evaluation trees and priced edges . . . 9
5.2 Frontier pricing kernels and guard-first rule . . . 9
5.3 Canonical debit and aggregation . . . 9
5.4 Axioms and proofs . . . 10
5.5 Notes on scope and implementation . . . 10

6 Complexity of Exact Δ_{fr} and Safe Approximations **10**
6.1 Decision problem (FRM-Dec) . . . 10
6.2 Strong NP-hardness (via 3-Partition) . . . 11
6.3 Safe upper-bounding algorithms . . . 11
6.4 Tightness regimes and exactness conditions . . . 12
6.5 Receipts and Loader checks . . . 12

7 Worked Examples **12**
7.1 Single M-axis + single S-axis (concrete $\mathbf{Q}$) . . . 12
7.2 Uncertainty lift on an M-axis: sub-Gaussian envelopes in $\mathbf{Q}'$ . . . 13
7.3 Intervals on an S-axis in $\mathbf{Q}'$: additive energy with bounds . . . 13
7.4 Budget-as-implication via residuation (concrete slice) . . . 13
7.5 Tiny Δ_{fr} instance with receipt . . . 14

8 Order, Monotonicity, and Acceptability **14**
8.1 Concrete acceptability on $\mathbf{Q}$ . . . 14
8.2 Uncertain acceptability on $\mathbf{Q}'$ . . . 15
8.3 Receipts and minimal checker . . . 16

9 Mechanization Hooks (Lean-style Signatures) **16**
9.1 Core structures (Lean-style) . . . 16
9.2 Builder from the Axis Catalog (JSON) . . . 17
9.3 Executable debit and receipts . . . 18
9.4 Minimal Loader checkers (trusted, tiny) . . . 18
9.5 Exported theorems (skim) . . . 19

10 Axis Catalog Bridge (Agent 2 Interop) **19**
10.1 Minimal JSON shape (normative) . . . 19
10.2 Validation and invariants . . . 20
10.3 Mapping to the product quantale and lift . . . 20
10.4 Kernel contract and receipts . . . 20
10.5 Example: two axes, mixed carriers . . . 21
10.6 Change management and compatibility . . . 21

11 Implementation Notes for Loader/Replay **21**
11.1 Runtime invariants (required) . . . 22
11.2 Pricing pipeline (single edge) . . . 22
11.3 Tree selection and approximations . . . 22
11.4 Receipts (normative shapes) . . . 23
11.5 Minimal Loader checks (trusted core) . . . 23
11.6 Failure modes and codes (non-exhaustive) . . . 24
11.7 Performance & numerics . . . 24

11.8 Security & hardening 24

12 Summary **24**

1 Preliminaries and Notation

Axes and typing. Let I be the finite (policy-declared) set of axes, partitioned into disjoint index sets S ("sum-like", cumulative) and M ("max-like", idempotent). The partition (S, M) comes from the Axis Catalog (Part 1, §10) and fixes each axis's aggregation law and uncertainty stance. We write

$$\mathbf{Q}_{\mathrm{S}} := \prod_{i \in S} Q_i^{\mathrm{S}}, \qquad \mathbf{Q}_{\mathrm{M}} := \prod_{j \in M} Q_j^{\mathrm{M}}, \qquad \mathbf{Q} := \mathbf{Q}_{\mathrm{M}} \times \mathbf{Q}_{\mathrm{S}}.$$

Here Q_i^{S} and Q_j^{M} are *complete carriers* (defined below) for the respective axes.

Carriers, orders, and distinguished elements. Each axis $i \in I$ is endowed with a complete lattice $(Q_i, \leq_i)$ with arbitrary joins $\bigvee$ and a *sup-continuous* monoid $(Q_i, \otimes_i, e_i)$ (the per-axis "sequential" aggregator). We reserve the symbol

$$\bot = \bot_i \in Q_i$$

for the lattice bottom on axis i. *Unknown is absorbing:* $x \otimes_i \bot = \bot \otimes_i x = \bot$ for all $x \in Q_i$. The concrete instantiations used throughout are (made formal in §3):

- $j \in M : Q_j = \overline{\mathbb{R}}$ with the usual order (joins are max); $\otimes_j = +$ (extended addition); $e_j = 0$; $\bot_j = -\infty$.
- $i \in S : Q_i = \overline{\mathbb{R}}_{\geq 0}$ with the usual order; $\otimes_i = +$; $e_i = 0$; $\bot_i = +\infty$ is *not* used—unknown is modeled as order-theoretic bottom via a dedicated sentinel bound in the complete slice (see §3).

For product notations we use the componentwise order on $\mathbf{Q}$:

$$(q_M, q_S) \leq (r_M, r_S) \iff \forall j \in M : (q_M)_j \leq_j (r_M)_j \;\text{ and }\; \forall i \in S : (q_S)_i \leq_i (r_S)_i.$$

We denote the product identity (for the per-axis $\otimes$) by $\mathbf{0} := (\mathbf{0}_{\mathrm{M}}, \mathbf{0}_{\mathrm{S}})$ with $\mathbf{0}_{\mathrm{M}} = (e_j)_{j \in M}$ and $\mathbf{0}_{\mathrm{S}} = (e_i)_{i \in S}$.

Typed projections and operators. We write $\pi_{\mathrm{M}}(q) = q_M \in \mathbf{Q}_{\mathrm{M}}$ and $\pi_{\mathrm{S}}(q) = q_S \in \mathbf{Q}_{\mathrm{S}}$. Part 2 uses two typed, coordinatewise operators (defined precisely in §3–2.1):

$$\begin{aligned} \text{hetero-join ("horizontal")} \quad & q \uplus r := (q_M \oplus r_M,\;\; q_S + r_S), \\ \text{sequential ("vertical")} \quad & q \star r := (q_M + r_M,\;\; q_S + r_S), \end{aligned}$$

where $\oplus$ is the idempotent max-join on $\mathbf{Q}_{\mathrm{M}}$ and $+$ is componentwise addition on the declared axes. These coincide with the product-quantale join and monoid (§3); see Remark 1.1 for cross-part notation.

Unknown, guards, and fail-closed discipline. Unknown/missing evidence is *uniformly* interpreted as the axis bottom $\bot$. It is absorbing for both $\uplus$ and $\star$ by construction (since both are built from per-axis $\otimes_i$ and joins). The Loader enforces a *guard-first* rule: if a priced input on any axis is $\bot$, the row is rejected *before* pricing; thus priced kernels never observe $\bot$. UI-level "missing" (sometimes described as a "top" sentinel) is *normalized* to $\bot$ prior to acceptance checks (Parts 1 & 3).

Dimensionless normalization. Frontier rows expose only normalized, dimensionless features to pricing kernels; unit rescalings in raw evidence cancel in normalization. Hence all prices (and therefore Δ_{fr}) are invariant to unit choices (formalized in §5).

Remark 1.1 (Typed structure and aliases)**.** We do not impose a single global semiring law on $\mathbf{Q}$. Instead, axes declare their laws, yielding a per-axis quantale; $\uplus$ and $\star$ are the product join and product monoid restricted by axis type. For cross-part consistency:

$$\text{Part 5's } \odot \equiv \uplus, \qquad \text{Part 1's product aggregator } (\wedge, \otimes) \equiv (\uplus, \star) \text{ here.}$$

Remark 1.2 (Residuation hook)**.** On any axis where $(Q_i, \otimes_i)$ is sup-continuous, the (right) residual $a \Rightarrow_i b := \bigvee\{x \in Q_i \mid a \otimes_i x \leq_i b\}$ exists and supports budget-as-implication/weakest-pre reasoning. We will use this in budget diffs and remaining-capacity calculations (§8).

Remark 1.3 (Notation summary)**.** We use boldface for product elements $\mathbf{0}, \mathbf{0}_{\mathrm{M}}, \mathbf{0}_{\mathrm{S}}$; $\bot$ for lattice bottom (absorbing); $\bigvee$ for arbitrary joins; $\biguplus$ for big heterogeneous joins. Projections are $\pi_{\mathrm{M}}, \pi_{\mathrm{S}}$. The extended reals $\overline{\mathbb{R}}$ are used where needed to ensure completeness and Scott-continuity of $+$ on directed joins.

2 Operations on Q

We now give the typed, product-style operators used throughout and prove the algebraic laws they satisfy. These operators coincide with the product-quantale join and monoid that will be constructed formally in §3.

Definition 2.1 (Typed aggregation operators)**.** For $q = (q_M, q_S)$, $r = (r_M, r_S) \in \mathbf{Q}$ define

$$\begin{aligned} \textit{hetero-join (horizontal)} \qquad & q \uplus r := \big(q_M \oplus r_M,\ \ q_S + r_S\big), \\ \textit{sequential (vertical)} \qquad & q \star r := \big(q_M + r_M,\ \ q_S + r_S\big). \end{aligned}$$

Here $\oplus$ is the componentwise max on $\mathbf{Q}_{\mathrm{M}}$ (idempotent), and $+$ is componentwise addition on declared axes. The distinguished identity is $\mathbf{0} := (\mathbf{0}_{\mathrm{M}}, \mathbf{0}_{\mathrm{S}})$ with $\mathbf{0}_{\mathrm{M}} = (0)_{j \in M}$ and $\mathbf{0}_{\mathrm{S}} = (0)_{i \in S}$. We write $\biguplus_{k \in K} q^{(k)}$ for the (finite) iterated $\uplus$.

Proposition 2.2 (Basic algebraic laws)**.** *For all $p, q, r \in \mathbf{Q}$ the following hold.*

(L1) ***Associativity/commutativity/idempotence of $\uplus$.*** *$(p \uplus q) \uplus r = p \uplus (q \uplus r)$, $q \uplus r = r \uplus q$, and $q \uplus q = q$.*

(L2) ***Associativity of $\star$.*** *$(p \star q) \star r = p \star (q \star r)$.*

(L3) ***Neutral element.*** *$q \uplus \mathbf{0} = \mathbf{0} \uplus q = q$ and $q \star \mathbf{0} = \mathbf{0} \star q = q$.*

(L4) ***Monotonicity (both arguments).*** *If $p \leq q$ and $r \leq s$ then $p \uplus r \leq q \uplus s$ and $p \star r \leq q \star s$.*

(L5) ***Absorbing bottoms (unknowns).*** *If any coordinate of q is $\bot$ then for all r, $q \uplus r = r \uplus q = \bot$ and $q \star r = r \star q = \bot$.*

Proof sketch. All statements are componentwise. On M-axes, $\oplus$ is idempotent, associative, and commutative; on S-axes, $+$ is associative and commutative. Identities follow from $\mathbf{0}_{\mathrm{M}} = \mathbf{0}$ for $+$ and the $\oplus$-identity $-\infty$ being encoded via the complete-slice construction, while $\mathbf{0}_{\mathrm{S}} = \mathbf{0}$ is the additive identity. Monotonicity is inherited from the coordinatewise monotonicity of max and $+$. Absorption holds since $\bot$ is the lattice bottom on each axis and is absorbing for the per-axis sequential monoid; both $\uplus$ and $\star$ are built from those per-axis operations. □

Remark 2.3 (Distributivity-on-demand)**.** On M-axes the vertical aggregator is $+$ and the horizontal aggregator is max. For all $a, b, c \in \overline{\mathbb{R}}$, $a + \max\{b, c\} = \max\{a + b,\ a + c\}$; hence the M-projection of $\star$ *distributes over* the M-projection of $\uplus$. Across S-axes both aggregators are $+$, so no cross-distribution law beyond associativity/commutativity is required. This matches the typed-product semantics: distribution only where the axis laws declare it.

Proposition 2.4 (Product alignment)**.** *Let* $(\mathbf{Q}, \leq, \bigvee, \otimes)$ *be the product-quantale constructed in §3. Then for all* $q, r \in \mathbf{Q}$*,*

$$q \uplus r \;=\; \bigvee\{q, r\} \quad \textit{(componentwise join)}, \qquad q \star r \;=\; q \otimes r \quad \textit{(componentwise monoid)}.$$

Proof sketch. By definition of the product construction, $\bigvee$ and $\otimes$ act coordinatewise; $\bigvee$ is max on M and $+$ on S; $\otimes$ is $+$ on both projections as declared by axis type. □

Remark 2.5 (Finite big-joins and budgets)**.** The notation $\biguplus_{e \in E} \mathrm{price}(e)$ used later in §5 is a finite $\uplus$-fold. By (L1)–(L3) it is well-defined up to permutation/parenthesization, enabling replayable receipts that only record multisets of priced edges and the associativity-neutral fold order.

Remark 2.6 (Residuation preview)**.** Whenever an axis i admits a right residual $a \Rightarrow_i b := \bigvee\{x \mid a \otimes_i x \leq_i b\}$ (e.g., when $\otimes_i$ is Scott-continuous), we obtain a *budget-as-implication* operator at the product level by taking residuals coordinatewise on the complete slice where they exist. This is used in §8 to compute remaining capacity and weakest-pre bounds.

3 Quantale Upgrade (per-axis completeness)

We formalize each axis as a complete, sup-continuous monoidal lattice (a quantale), and then take a product construction. This supplies completeness (arbitrary joins), order-theoretic reasoning, and hooks such as residuation on the slices where it exists. The *typed* runtime operators from §2 are then recovered by projecting to M- vs. S-axes (Remark 3.5).

Definition 3.1 (Axis quantale)**.** For an axis i, an *axis quantale* is a tuple

$$\mathcal{Q}_i := (Q_i,\ \leq_i,\ \bigvee_i,\ \otimes_i,\ e_i,\ \bot_i)$$

such that:

(a) $(Q_i, \leq_i)$ is a complete lattice with arbitrary joins $\bigvee_i$.

(b) $(Q_i, \otimes_i, e_i)$ is a monoid.

(c) $\otimes_i$ is *sup-continuous* (Scott-continuous) in each argument: for all $a \in Q_i$ and $S \subseteq Q_i$,

$$a \otimes_i \bigvee_i S = \bigvee_i \{a \otimes_i s : s \in S\}, \qquad \bigvee_i S \otimes_i a = \bigvee_i \{s \otimes_i a : s \in S\}.$$

(d) $\bot_i$ is the lattice bottom and is *absorbing*: $x \otimes_i \bot_i = \bot_i \otimes_i x = \bot_i$ for all $x \in Q_i$.

Assumption 3.2 (Policy-instantiated slices)**.** For $j \in M$ we take $Q_j = \overline{\mathbb{R}}$ with the usual order, $\bigvee_j = \max$, $\otimes_j = +$ (extended addition), $e_j = 0$, $\bot_j = -\infty$. For $i \in S$ we take the complete slice $Q_i = \{\bot_i\} \uplus \overline{\mathbb{R}}_{\geq 0}$ ordered with $\bot_i$ least and the usual order on $\overline{\mathbb{R}}_{\geq 0}$; joins are numeric sups lifted with $\bot_i$, and $\otimes_i$ extends $+$ by absorption at $\bot_i$.

Theorem 3.3 (Product quantale)**.** *Let $I = M \cup S$ and suppose each $i \in I$ carries an axis quantale $\mathcal{Q}_i$ as in Definition 3.1. Then the Cartesian product*

$$\mathbf{Q}^{\vee} := \prod_{i \in I} Q_i$$

equipped with the componentwise order $\leq$, componentwise joins $\bigvee$, componentwise monoid $\otimes$ with identity $e = (e_i)_{i \in I}$, and bottom $\bot = (\bot_i)_{i \in I}$, is a (unital) quantale.

Proof sketch. Products of complete lattices are complete under the pointwise order; their joins compute pointwise. The monoid product is pointwise associative with identity e. Sup-continuity/distributivity of $\otimes$ over arbitrary joins holds pointwise by assumption for each coordinate. Absorption of $\bot$ follows from coordinatewise absorption of $\bot_i$. □

Corollary 3.4 (Residuation on complete slices)**.** *If an axis i admits a right residual $a \Rightarrow_i b := \bigvee_i \{x \in Q_i \mid a \otimes_i x \leq_i b\}$ (i.e., $\otimes_i$ is sup-continuous), then the product admits the coordinatewise residual on the complete subproduct where all participating axes have residuals.*

Remark 3.5 (Typed operators vs. lattice join)**.** The quantale $(\mathbf{Q}^{\vee}, \bigvee, \otimes)$ provides *lattice* joins and the sequential monoid. Our runtime "heterogeneous" operator $\uplus$ from §2 is *typed* by design:

$$\pi_{\mathrm{M}}(q \uplus r) \;=\; \pi_{\mathrm{M}}(q) \bigvee \pi_{\mathrm{M}}(r) \quad \text{(join on } M\text{)}, \qquad \pi_{\mathrm{S}}(q \uplus r) \;=\; \pi_{\mathrm{S}}(q) \otimes \pi_{\mathrm{S}}(r) \quad \text{(monoid on } S\text{)}.$$

Thus $\uplus$ *agrees with the lattice join on M* and with the *monoid* on S. The sequential operator $\star$ is the product monoid on both projections. This is the "distributivity-on-demand" policy: we only require distribution where the axis law declares it (e.g., $+$ distributes over max on M).

Remark 3.6 (Unknown as absorbing bottom, globally)**.** Since each axis bottom $\bot_i$ is absorbing for $\otimes_i$, the product bottom $\bot$ is absorbing for the product monoid and, by Remark 3.5, for both runtime operators $\uplus$ and $\star$. Guard-first enforcement ensures priced kernels never observe $\bot$; rows with $\bot$ on any priced coordinate are rejected before pricing.

4 Uncertainty-aware Lift $\mathbf{Q}'$

We replace scalar coordinates with order-theoretic *carriers of uncertainty* and lift the operators from §2 in a way that is sound with respect to the concrete algebra $\mathbf{Q}$. The lift is phrased via abstract interpretation (*Galois connections*) so that adding new carriers is principled and local.

4.1 Carriers and orders (with absorbing unknown)

For each axis $i \in I$ choose a base abstract domain D_i (intervals, sub-Gaussian envelopes, robust sets, ...) with a partial order $\preceq_i$ where *more conservative* (wider, riskier) elements are *larger*. We extend each domain with an explicit absorbing bottom $\bot_i$ (fail-closed unknown) by disjoint sum:

$$Q_i' \;:=\; \{\bot_i\} \;+\; D_i, \qquad \bot_i \;\preceq_i\; d \;\; (\forall d \in D_i).$$

Operations on Q_i' are absorbing at $\bot_i$ and otherwise act inside D_i.

Canonical base domains. We instantiate D_i from the catalog as follows (representative choices):

(C1) *Intervals* on $\overline{\mathbb{R}}$: $D_i = \{[\ell, u] : \ell \le u\}$. Order: $[l_1, u_1] \preceq_i [l_2, u_2] \iff [l_2, u_2] \subseteq [l_1, u_1]$ (*reverse inclusion*). Closed operations:

$$[l_1, u_1] \ \oplus_i \ [l_2, u_2] = [\max\{l_1, l_2\},\ \max\{u_1, u_2\}],$$
$$[l_1, u_1] \ \otimes_i \ [l_2, u_2] = [l_1 + l_2,\ u_1 + u_2].$$

(C2) *Sub-Gaussian envelopes*: $D_i = \{(\mu, \sigma^2) : \sigma^2 \ge 0\}$. Order: coordinatewise $(\mu, \sigma^2) \preceq_i (\mu', \sigma'^2) \iff \mu \le \mu' \ \wedge \ \sigma^2 \le \sigma'^2$. Closed operations:

$$(\mu, \sigma^2) \ \oplus_i \ (\mu', \sigma'^2) := (\max\{\mu, \mu'\},\ \max\{\sigma^2, \sigma'^2\}), \quad (\mu, \sigma^2) \ \otimes_i \ (\mu', \sigma'^2) := (\mu + \mu',\ \sigma^2 + \sigma'^2).$$

(C3) *Robust sets* in $\mathbb{R}^d$: $D_i = \{U \subseteq \mathbb{R}^d : U \text{ nonempty, compact, convex}\}$. Order: reverse inclusion $U_1 \preceq_i U_2 \iff U_2 \subseteq U_1$. Closed operations:

$$U_1 \ \oplus_i \ U_2 := \mathrm{conv}(U_1 \cup U_2), \qquad U_1 \ \otimes_i \ U_2 := U_1 \oplus_{\mathrm{Mink}} U_2.$$

In all cases extend by absorption: $\bot_i \oplus_i d = \bot_i$, $\bot_i \otimes_i d = \bot_i$ (and symmetrically). Intuitively, $\bot_i$ denotes "anything could happen" and concretizes to the full concrete carrier, ensuring fail-closed composition.

4.2 Abstract interpretation interface

Let the concrete carrier on axis i be Q_i (from §3) and its powerset $(\mathcal{P}(Q_i), \subseteq)$. We connect the concrete and abstract worlds by *concretization* and *abstraction* maps:

$$\gamma_i : Q_i' \to \mathcal{P}(Q_i), \quad \alpha_i : \mathcal{P}(Q_i) \to Q_i', \quad \text{with} \quad \alpha_i \dashv \gamma_i \ \ \text{(Galois connection)}.$$

We require $\gamma_i(\bot_i) = Q_i$. For the bases above:

- Intervals: $\gamma_i([\ell, u]) = \{x \in Q_i : \ell \le x \le u\}$, and α_i maps a nonempty set to its tightest interval hull; $\alpha_i(\emptyset)$ is defined as the least informative element (absorbing $\bot_i$).
- Sub-Gaussian: $\gamma_i(\mu, \sigma^2)$ the set of distributions dominated by this envelope; α_i maps a family of distributions to a safe upper envelope.
- Robust sets: $\gamma_i(U) = U$, $\alpha_i(S) = \mathrm{cl}\,\mathrm{conv}(S)$.

Definition 4.1 (Sound local lift)**.** Binary operations $\oplus_i, \otimes_i$ on Q_i' are a *sound lift* of the concrete $\bigvee_i, \otimes_i$ iff for all $x', y' \in Q_i'$:

$$\{\, x \bigvee_i y : x \in \gamma_i(x'),\ y \in \gamma_i(y') \,\} \ \subseteq \ \gamma_i(x' \oplus_i y'), \quad \{\, x \otimes_i y : x \in \gamma_i(x'),\ y \in \gamma_i(y') \,\} \ \subseteq \ \gamma_i(x' \otimes_i y').$$

Lemma 4.2 (Sound lift to the product)**.** *If (α_i, γ_i) form a Galois connection and $\oplus_i, \otimes_i$ are sound lifts for all $i \in I$, then the product $\mathbf{Q}' := \prod_{i \in I} Q_i'$ with componentwise order, $\oplus := \prod \oplus_i$, and $\otimes := \prod \otimes_i$ is a sound abstraction of $(\mathbf{Q}^\vee, \bigvee, \otimes)$. In particular, the algebraic laws of §2 (monotonicity, associativity, identities, absorption of $\bot$) hold verbatim in $\mathbf{Q}'$.*

Proof sketch. Soundness and monotonicity compose coordinatewise; absorption follows since each $\bot_i$ is absorbing and $\gamma_i(\bot_i) = Q_i$. Associativity/identities are preserved by sound local lifts on complete carriers. □

4.3 Typed operators on $\mathbf{Q}'$

We reuse the runtime operators $\uplus, \star$ from §2 *verbatim*, now acting on $\mathbf{Q}'$ coordinatewise via $\oplus_i, \otimes_i$:

$$q' \uplus r' := (\ \pi_{\mathrm{M}}(q') \oplus \pi_{\mathrm{M}}(r'),\quad \pi_{\mathrm{S}}(q') \otimes \pi_{\mathrm{S}}(r')\ ),$$
$$q' \star r' := (\ \pi_{\mathrm{M}}(q') \otimes \pi_{\mathrm{M}}(r'),\quad \pi_{\mathrm{S}}(q') \otimes \pi_{\mathrm{S}}(r')\ ).$$

By Definition 4.2, these are sound lifts of the concrete product join and monoid (§3).

4.4 Acceptance stances and upward closure

Let $B' \in \mathbf{Q}'$ be a budget and $q' \in \mathbf{Q}'$ a reported quantity. We reuse Part 1's acceptance stances (probabilistic / robust / hybrid), applied *axiswise* to the carriers in Q'_i. Formally:

Definition 4.3 (Axiswise acceptance)**.** Fix stance parameters (e.g. quantile $1-\delta$ on risk axes). We say q' is *acceptable* under B' if for all $i \in I$, $q'_i \preceq_i B'_i$ where $\preceq_i$ is the carrier's conservative order induced by the stance (e.g. reverse inclusion for intervals, envelope dominance for sub-Gaussian pairs).

Proposition 4.4 (Upward-closed acceptability)**.** *Acceptance is upward-closed in $\mathbf{Q}'$: if $q' \preceq r'$ and r' is acceptable w.r.t. B', then q' is acceptable w.r.t. B'.*

Proof. Immediate from axiswise monotonicity of $\preceq_i$. □

Remark 4.5 (Guard lifting)**.** Guard-first discipline (§2) lifts to $\mathbf{Q}'$: any priced coordinate with value $\bot_i$ triggers rejection before pricing. Since $\gamma_i(\bot_i) = Q_i$, no acceptance rule can *force* an acceptance through unknowns.

4.5 Worked example: sub-Gaussian acceptance

Consider an M-axis with sub-Gaussian envelope carrier (μ, σ^2) and a *probabilistic* stance at confidence $1-\delta$. A sound upper bound on the $(1-\delta)$-quantile is $q_{1-\delta} \le \mu + z_{1-\delta}\,\sigma$ where $z_{1-\delta}$ is the standard normal quantile and $\sigma = \sqrt{\sigma^2}$. Thus $x' = (\mu, \sigma^2)$ is acceptable under budget $B' = (\mu_B, \sigma_B^2)$ iff

$$\mu + z_{1-\delta}\,\sqrt{\sigma^2} \ \le\ \mu_B + z_{1-\delta}\,\sqrt{\sigma_B^2}.$$

Monotonicity of $z_{1-\delta}$ and $\sqrt{\cdot}$ implies upward-closed acceptance. Composition laws are closed:

$$(\mu, \sigma^2) \oplus (\mu', \sigma'^2) = (\max\{\mu, \mu'\}, \max\{\sigma^2, \sigma'^2\}), \quad (\mu, \sigma^2) \otimes (\mu', \sigma'^2) = (\mu + \mu', \sigma^2 + \sigma'^2),$$

matching $\uplus$ (max-like) and $\star$ (sequential sum-like) projections.

4.6 Interoperability and receipts

- **Axis catalog is the source of truth.** The catalog declares the carrier type and stance (per axis), from which D_i, $\preceq_i$, $\oplus_i, \otimes_i$, and (α_i, γ_i) are fixed. The product $\mathbf{Q}'$ is thereby *fully determined*.
- **Receipts.** Acceptance checks produce axiswise witnesses (e.g., interval containments, quantile inequalities) bundled into receipts for Loader replay. Since checks are axiswise and monotone, verification is lightweight.

5 Frontier Debit Δ_{fr}: Canonical Construction

We now give the constructive, runtime-aligned definition of the frontier debit and prove the axioms it must satisfy. The definition is parameterized by the policy π (which determines which boundaries are priced and how) and by the Loader's admissibility constraints (acyclic order, EnvLock replay).

5.1 Admissible evaluation trees and priced edges

Let $\mathcal{A}$ be a multiset of primitive evaluation actions (adapter calls, kernel invocations, etc.). An *evaluation tree* over $\mathcal{A}$ is a full binary tree T whose leaves are in bijection with $\mathcal{A}$ and whose internal nodes denote sequential compositions. We write $\mathcal{T}_{\mathrm{adm}}(\mathcal{A})$ for the set of *admissible* trees: those whose in-order evaluations respect the Loader's acyclicity and EnvLock discipline (Parts 3–4), so that every $T \in \mathcal{T}_{\mathrm{adm}}$ is deterministically replayable.

For a fixed policy π, each internal edge e of T *may* be declared a *frontier* (priced boundary) by π. Let $E_\pi(T) \subseteq E(T)$ be the multiset of such priced edges.

5.2 Frontier pricing kernels and guard-first rule

Each priced edge $e \in E_\pi(T)$ carries a *frontier row* (normalized, dimensionless fields) that is fed to a policy-selected *pricing kernel* $\mathrm{price}_\pi(e)$ with the following contract:

$\mathrm{price}_\pi(e) \in \mathbf{Q}_{\mathrm{M}} \times \{0_{\mathbf{Q}_{\mathrm{S}}}\}$, price_π monotone in its inputs, dimensionless, and nonnegative/idempotent by axis.

Guard-first (fail-closed) enforcement. If any priced input is $\bot$ on any axis, the Loader *rejects the row before* calling price_π. Thus pricing kernels never observe $\bot$. This aligns algebraic absorption (Remark 3.6) with runtime safety.

5.3 Canonical debit and aggregation

Given an admissible tree T, the (typed) aggregate price of its priced edges is the heterogeneous fold in $\mathbf{Q}$ (max on M, sum on S; S-part is pinned at 0):

$$\Delta_\pi(T) \;:=\; \biguplus_{e \in E_\pi(T)} \mathrm{price}_\pi(e) \;\in\; \mathbf{Q}_{\mathrm{M}} \times \{0_{\mathbf{Q}_{\mathrm{S}}}\}, \qquad \biguplus \equiv \uplus \text{ from §2.} \tag{1}$$

The *canonical frontier debit* takes the best admissible reassociation:

$$\Delta_{\mathrm{fr}}(\mathcal{A}, \pi) \;:=\; \bigwedge_{T \in \mathcal{T}_{\mathrm{adm}}(\mathcal{A})} \Delta_\pi(T) \;\in\; \mathbf{Q}_{\mathrm{M}} \times \{0_{\mathbf{Q}_{\mathrm{S}}}\}. \tag{2}$$

The infimum is taken in the product order on $\mathbf{Q}$, ensuring axiswise safety.

Remark 5.1 (Replay receipts)**.** By (L1)–(L3), $\uplus$ is associative/commutative/idempotent on M and additive on S, so $\Delta_\pi(T)$ is invariant to the fold order. Receipts therefore encode the multiset $E_\pi(T)$ (with stable identifiers) and the kernel outputs, sufficient for Loader re-checks without resynthesizing T.

5.4 Axioms and proofs

We restate and prove the frontier axioms (Part 1, §6) in this construction.

Proposition 5.2 (Frontier axioms)**.** *Fix $\mathcal{A}$ and policy π. Then:*

(F1) ***Identity.*** *If $E_\pi(T) = \varnothing$ for some (hence all) T, then $\Delta_{\mathrm{fr}}(\mathcal{A}, \pi) = \mathbf{0}_{\mathrm{M}}$ on M and 0 on S.*

(F2) ***Nonnegativity/idempotence.*** *On S-axes, $(\Delta_{\mathrm{fr}})_S = 0$. On M-axes, $(\Delta_{\mathrm{fr}})_M \geq \mathbf{0}_{\mathrm{M}}$ and respects idempotence (joins do not double-charge repeats).*

(F3) ***Subadditivity.*** *For disjoint action-multisets $\mathcal{A}_1, \mathcal{A}_2$,*

$$\Delta_{\mathrm{fr}}(\mathcal{A}_1 \cup \mathcal{A}_2, \pi) \;\leq\; \Delta_{\mathrm{fr}}(\mathcal{A}_1, \pi) \;\uplus\; \Delta_{\mathrm{fr}}(\mathcal{A}_2, \pi).$$

(F4) ***Policy monotonicity.*** *If $\pi \preceq \pi'$ (every price under π is $\leq$ its counterpart under π', and/or $E_\pi(T) \subseteq E_{\pi'}(T)$ for all T), then $\Delta_{\mathrm{fr}}(\mathcal{A}, \pi) \leq \Delta_{\mathrm{fr}}(\mathcal{A}, \pi')$.*

(F5) ***Dimensionless invariance.*** *Consistent rescaling of physical units in raw evidence leaves Δ_{fr} unchanged (pricing kernels consume only normalized fields).*

Proof sketch. (F1) If no edges are priced, (1) is the empty $\uplus$-fold, which yields $(\mathbf{0}_{\mathrm{M}}, 0)$; the infimum preserves this value. (F2) The S-projection is identically 0 by the codomain of price_π. The M-projection is a finite max-fold of nonnegative, idempotent entries, bounded below by $\mathbf{0}_{\mathrm{M}}$. (F3) Let T_1, T_2 be (near-)optimal admissible trees for $\mathcal{A}_1, \mathcal{A}_2$. The admissible tree T formed by sequentially composing T_1 and T_2 exists by Loader acyclicity; then $\Delta_\pi(T) = \Delta_\pi(T_1) \uplus \Delta_\pi(T_2)$ by (1) and §2. Taking infima over supersets gives the claim. (F4) Componentwise monotonicity of $\uplus$ and the kernel together with $E_\pi(T) \subseteq E_{\pi'}(T)$ imply $\Delta_\pi(T) \leq \Delta_{\pi'}(T)$ for all T; infima preserve the inequality. (F5) Kernels operate on dimensionless inputs; rescalings cancel during normalization, so each $\mathrm{price}_\pi(e)$ is invariant. Aggregation and infimum preserve invariance. □

5.5 Notes on scope and implementation

- **Axis isolation.** $\mathrm{price}_\pi(e)$ must write only to declared axes. Monotone homomorphisms from frontier rows to axes are checked axiswise.
- **Unknowns.** Any $\perp$ in priced inputs triggers guard failure (pre-pricing). Post-pricing algebra retains absorption to prevent downstream cancellation by ignorance (§2, §4).
- **Typed product.** By §3, $\uplus$ and $\star$ are product join/monoid on the appropriate projections, so the debit algebra inherits completeness and distributivity-on-demand automatically.

6 Complexity of Exact Δ_{fr} and Safe Approximations

We formalize the decision version of computing Δ_{fr}, prove *strong* NP-hardness by reduction from 3-Partition, and specify sound, receipt-carrying upper-bounding algorithms used by the Loader.

6.1 Decision problem (FRM-Dec)

Fix an action multiset $\mathcal{A}$, a policy π, admissible trees $\mathcal{T}_{\mathrm{adm}}(\mathcal{A})$ (Loader-enforced), and pricing kernels price_π (§5). For any $T \in \mathcal{T}_{\mathrm{adm}}(\mathcal{A})$, let $\Delta_\pi(T)$ be as in (1). The canonical debit is $\Delta_{\mathrm{fr}}(\mathcal{A}, \pi) = \bigwedge_T \Delta_\pi(T)$ as in (2).

Definition 6.1 (FRM-Dec: Frontier Reassociation Minimization (decision))**.** Given threshold $K \in \mathbf{Q}_{\mathrm{M}} \times \{0_{\mathbf{Q}_{\mathrm{S}}}\}$, does there exist an admissible $T \in \mathcal{T}_{\mathrm{adm}}(\mathcal{A})$ such that $\Delta_\pi(T) \le K$?

Membership in NP is immediate: a certificate is a concrete T; the Loader can verify admissibility, evaluate kernels on $E_\pi(T)$, fold them by $\uplus$ (§2), and check $\Delta_\pi(T) \le K$ in time polynomial in $|\mathcal{A}|$.

6.2 Strong NP-hardness (via 3-Partition)

We show hardness *even with a single M-axis*, arity-3 grouping, and a simple overflow kernel.

Theorem 6.2 (Strong NP-hardness of FRM-Dec)**.** *FRM-Dec is strongly NP-hard. The hardness holds for instances with: (i) one M-axis, (ii) policy π restricting legal internal merges to groups of size exactly 3, and (iii) a kernel on each priced edge e returning* $\mathrm{price}_\pi(e) = \max\{0,\, S_e - T\}$ *where S_e is the sum of the three leaf weights in the group under e and $T > 0$ is fixed.*

Proof sketch. Reduce from 3-Partition (strongly NP-complete). Given multiset $S = \{s_1, \dots, s_{3m}\}$ of integers with $\sum S = mT$ and $T/4 < s_k < T/2$ for all k, build $\mathcal{A}$ with one leaf of weight s_k per element. Policy π allows only merges that create groups of size 3 and declares one priced edge per group. The kernel price is the overflow $\max\{0, \sum_{r \in G} s_r - T\}$. Since $\uplus$ on the single M-axis is max, the aggregated debit for a tree is the maximum group overflow. Set $K = 0$. Then there exists T with $\Delta_\pi(T) \le 0$ iff the multiset can be partitioned into m triples each summing exactly T, i.e., iff the 3-Partition instance is a yes-instance. The construction is polynomial and independent of numeric encoding size (strong hardness). □

6.3 Safe upper-bounding algorithms

Because exact optimization is intractable in general, the Loader admits algorithms that produce a *feasible* tree $\widehat{T}$ (or a value derived from one) and report the resulting $\widehat{\Delta}_{\mathrm{fr}} := \Delta_\pi(\widehat{T})$. Feasibility ensures $\widehat{\Delta}_{\mathrm{fr}}$ is an *upper bound* on Δ_{fr} (a minimization).

A1. Greedy local pairing (linearithmic). Maintain a priority queue of candidate merges; repeatedly merge the pair whose priced edge yields the smallest marginal increase in $\uplus$-fold on M (ties broken deterministically). Complexity $O(n \log n)$ with a suitable heap; outputs a tree $\widehat{T}_{\mathrm{greedy}}$ and bound $\widehat{\Delta}_{\mathrm{fr}}^{\mathrm{greedy}} = \Delta_\pi(\widehat{T}_{\mathrm{greedy}})$.

A2. Bounded-width dynamic programming (polytime for fixed w). Restrict intermediate clusters to size $\le w$ (or, equivalently, operate on instances whose association graph has treewidth $\le w$). DP over partitions of size at most w yields time $O(n^{w+1})$ and space $O(n^w)$. Exact on treewidth-w families; otherwise produces a feasible $\widehat{T}_w$ with bound $\widehat{\Delta}_{\mathrm{fr}}^{(w)}$.

A3. Convex/robust relax-and-round (with receipts). Formulate a 0–1 assignment/association problem whose feasible vertices are legal trees or groupings. Relax to LP/SOCP (for piecewise-linear or conic kernels) to obtain a fractional solution with value L (a *lower* bound on the true optimum). Apply a *monotone rounding* that constructs a legal $\widehat{T}$ such that $\Delta_\pi(\widehat{T}) \ge L$. Report the *feasible* value $\widehat{\Delta}_{\mathrm{fr}}^{\mathrm{rr}} := \Delta_\pi(\widehat{T})$ as the upper bound, and include a receipt enabling the Loader to verify: (i) the relaxed instance digest, (ii) the reported L, (iii) legality of $\widehat{T}$ under π, and (iv) $\Delta_\pi(\widehat{T}) \ge L$ by direct fold.

Theorem 6.3 (Safe approximations)**.** *For A1–A3, the reported value is a sound upper bound:* $\Delta_{\mathrm{fr}}(\mathcal{A}, \pi) \le \widehat{\Delta}_{\mathrm{fr}}$.

Proof sketch. A1/A2 produce feasible $\widehat{T}$, so $\Delta_{\mathrm{fr}} = \bigwedge_T \Delta_\pi(T) \le \Delta_\pi(\widehat{T}) = \widehat{\Delta}_{\mathrm{fr}}$. For A3, the Loader verifies feasibility and recomputes $\Delta_\pi(\widehat{T})$; the same inequality applies. (The relaxed value L is a separate *lower* bound retained for diagnostics.) □

6.4 Tightness regimes and exactness conditions

- **Greedy (A1).** Optimal on rank-1 metrics and certain convex, submodular kernel families; typically tight in *nearly decomposable* instances.
- **Bounded-width DP (A2).** Exact when the association graph has treewidth $\le w$. In practice, Loader can apply A2 within windows or along separators and stitch bounds via $\uplus$.
- **Relax-and-round (A3).** Exact when the relaxed polytope is integral (e.g., totally unimodular structures) or when kernels are piecewise-linear convex over network-flow style constraints. Robust variants remain valid under carrier uncertainty by lifting epigraphs axiswise (§4).

6.5 Receipts and Loader checks

Any reported $\widehat{\Delta}_{\mathrm{fr}}$ must be accompanied by a minimal receipt:

1. *Tree witness:* the legal $\widehat{T}$ (or grouping) with stable leaf/edge IDs.
2. *Kernel outputs:* the multiset $\{\mathrm{price}_\pi(e)\}_{e \in E_\pi(\widehat{T})}$ or the raw rows so the Loader can recompute them.
3. *Fold/value:* the $\uplus$-fold result (and, for A3, the relaxed value L).

The Loader deterministically rechecks admissibility, recomputes prices and the fold, and verifies (optional) $L \le \widehat{\Delta}_{\mathrm{fr}}$. This yields a replayable, byte-level proof that the reported value is a *safe* upper bound, satisfying the safety-or-halt contract.

7 Worked Examples

We illustrate the typed operators, the uncertainty lift, acceptance, and frontier accounting with concrete, small instances. Throughout, "unknown" is the absorbing bottom $\bot$ and *priced rows with* $\bot$ are rejected pre-pricing by the Loader's guard-first rule (§5).

7.1 Single M-axis + single S-axis (concrete Q)

Let $M = \{\textsc{Risk}\}$ with carrier $\overline{\mathbb{R}}$, and $S = \{\textsc{Energy}\}$ with carrier $\overline{\mathbb{R}}_{\ge 0}$. Take three primitive actions a, b, c with frontier prices (dimensionless):

$$\mathrm{price}(a) = (0.06,\, 0), \quad \mathrm{price}(b) = (0.09,\, 0), \quad \mathrm{price}(c) = (0.04,\, 0),$$

so the M-projection is a max-like risk contribution and the S-projection is pinned at 0 (as required by §5). Consider two admissible trees T_1, T_2 that differ only in grouping; both expose exactly the same priced edges but in different locations.

Using $\uplus$ (max on M, $+$ on S):

$$\Delta(T_1) = \biguplus\{\mathrm{price}(a), \mathrm{price}(b), \mathrm{price}(c)\} = (\max\{0.06, 0.09, 0.04\},\, 0) = (0.09,\, 0).$$

Re-associating doesn't change the multiset or the fold value (Prop. 2.2), hence $\Delta(T_2) = (0.09, 0)$ as well, and $\Delta_{\text{fr}} = \bigwedge\{(0.09, 0), (0.09, 0)\} = (0.09, 0)$. If the policy priced only the edge incident to $\{a, c\}$ (idempotent join on M):

$$\Delta(T_1) = \biguplus\{\text{price}(a), \text{price}(c)\} = (\max\{0.06, 0.04\}, 0) = (0.06, 0), \quad \Delta_{\text{fr}} = (0.06, 0).$$

Unknowns. If price(b) were fed an $\bot$ field, the row would be guard-rejected and not appear in the fold; post-pricing absorption still prevents any downstream cancellation by ignorance.

7.2 Uncertainty lift on an M-axis: sub-Gaussian envelopes in $\mathbf{Q}'$

Let RISK use the carrier (μ, σ^2) and a probabilistic stance at confidence $1 - \delta$ (§4.4). Two priced edges yield

$$x' = (\mu_1, \sigma_1^2) = (0.030,\ 0.0025), \qquad y' = (\mu_2, \sigma_2^2) = (0.050,\ 0.0016).$$

Typed operators on $\mathbf{Q}'$ (§4.3) give

$$x' \uplus y' = (\max\{0.030, 0.050\},\ \max\{0.0025, 0.0016\}) = (0.050,\ 0.0025), \quad x' \star y' = (0.080,\ 0.0041).$$

Acceptance against budget $B' = (\mu_B, \sigma_B^2) = (0.070,\ 0.0036)$ at level $1 - \delta$ requires

$$\mu + z_{1-\delta}\sqrt{\sigma^2} \ \le\ \mu_B + z_{1-\delta}\sqrt{\sigma_B^2}.$$

Both $x' \uplus y'$ and $x' \star y'$ satisfy the inequality for any fixed $z_{1-\delta} \ge 0$ because $0.050 \le 0.070$ and $\sqrt{0.0025} = 0.05 \le \sqrt{0.0036} = 0.06$, and $0.080 \le 0.070 + (0.06 - 0.05) z_{1-\delta}$ holds whenever the budget's quantile margin dominates the mean gap (axiswise and stance-parameter dependent). Upward-closed acceptance (§4.4) ensures tightening B' shrinks the accepted set monotonically.

7.3 Intervals on an S-axis in $\mathbf{Q}'$: additive energy with bounds

Let ENERGY use interval carriers with reverse-inclusion order and Minkowski sum. Suppose two stages have interval estimates $E_1' = [9, 11]$ kJ, $E_2' = [4, 6]$ kJ. Then

$$E_1' \star E_2' = [13, 17]\ \text{kJ}, \qquad E_1' \uplus E_2' = [\max\{9, 4\},\ \max\{11, 6\}] = [9, 11]\ \text{kJ}.$$

Here $\uplus$ on S-axes is the monoid (sum-carrier law) by Remark 3.5; thus sequential composition grows the interval via Minkowski sum while the horizontal aggregator reflects alternative paths (coordinatewise upper hull). Against a budget $B_E' = [0, 15]$ kJ, the sequential result is *not* acceptable ($[13, 17] \not\preceq [0, 15]$), while the horizontal alternative $[9, 11]$ is acceptable.

7.4 Budget-as-implication via residuation (concrete slice)

On any axis with sup-continuous $+$ (e.g., ENERGY), the residual computes remaining capacity (§3):

$$a \Rightarrow b \ =\ \bigvee\{x \mid a + x \le b\} \ =\ b - a \quad (\text{clamped at } \bot \text{ if } a > b).$$

With budget $B_E = 15$ and incurred cost $C_E = 9.7$, we have $C_E \Rightarrow B_E = 5.3$. If any input were $\bot$, the residual is $\bot$ (fail-closed), and the Loader would reject prior to pricing.

7.5 Tiny Δ_{fr} instance with receipt

Consider four leaves with weighted rows on a single M-axis: $s = \{4, 7, 5, 6\}$, policy π prices every internal edge as the overflow $\max\{0, \mathrm{sum} - T\}$ with $T = 10$ (as in Thm. 6.2). Two admissible trees:

$$\begin{array}{ll} T_A: & (4{+}7) \text{ vs } (5{+}6) \quad \text{group sums} = 11, 11 \Rightarrow \text{edge prices} = 1, 1 \Rightarrow \Delta_\pi(T_A) = \max\{1, 1\} = 1. \\ T_B: & (4{+}6) \text{ vs } (7{+}5) \quad \text{group sums} = 10, 12 \Rightarrow \text{edge prices} = 0, 2 \Rightarrow \Delta_\pi(T_B) = \max\{0, 2\} = 2. \end{array}$$

Hence $\Delta_{\mathrm{fr}} = \bigwedge\{1, 2\} = 1$. A minimal replay receipt includes: (i) the chosen tree T_A (stable leaf IDs), (ii) the two group rows and kernel outputs $1, 1$, (iii) the $\uplus$-fold $\max\{1, 1\} = 1$. The Loader re-check recomputes both prices, folds by $\uplus$, and confirms the bound (§6.5). If any priced row had $\bot$, the receipt would be invalidated at guard time and the run halted.

These examples exercise: (1) typed aggregation and idempotence on M; (2) sound uncertainty lifting and stance-based acceptance; (3) residuation for remaining capacity; and (4) replayable, upper-bounding frontier accounting consistent with the Loader's safety-or-halt contract.

8 Order, Monotonicity, and Acceptability

We formalize budget checking and its algebra, first on the concrete product $\mathbf{Q}$ and then for the uncertainty lift $\mathbf{Q}'$. Throughout, Δ_{fr} is the canonical frontier debit from §5 and the operators $\uplus, \star$ are as in §2. Recall that priced kernels write only to M-axes, hence $(\Delta_{\mathrm{fr}})_S = 0$ (§5).

8.1 Concrete acceptability on Q

Definition 8.1 (Acceptability)**.** Given a budget $B \in \mathbf{Q}$ and a reported quantity $q \in \mathbf{Q}$, we say that q is *acceptable against* B (write $\mathrm{Acc}_{\mathbf{Q}}(q; B)$) iff

$$\Delta_{\mathrm{fr}} \ \uplus \ q \ \leq \ B.$$

Axiswise, this is the product order on $\mathbf{Q}$.

Since $\uplus$ is max on M and $+$ on S (Remark 3.5), we have explicit axis criteria.

Proposition 8.2 (Axiswise acceptance conditions)**.** *Let $B = (B_M, B_S)$, $q = (q_M, q_S)$. Then*

$$\mathrm{Acc}_{\mathbf{Q}}(q; B) \quad \Longleftrightarrow \quad \begin{cases} M\text{-axes:} & \max\big((\Delta_{\mathrm{fr}})_M, \ q_M\big) \ \leq \ B_M, \\ S\text{-axes:} & (\Delta_{\mathrm{fr}})_S + q_S \ \leq \ B_S. \end{cases}$$

In particular, because $(\Delta_{\mathrm{fr}})_S = 0$ by construction, the S-criterion reduces to $q_S \leq B_S$.

Proof. Unfold the definition $\Delta_{\mathrm{fr}} \uplus q = (\max\{(\Delta_{\mathrm{fr}})_M, q_M\}, \ (\Delta_{\mathrm{fr}})_S + q_S)$ and compare componentwise to B. □

Proposition 8.3 (Monotone admissibility)**.** *Fix Δ_{fr}. If $q \leq r$ and $\mathrm{Acc}_{\mathbf{Q}}(r; B)$ holds, then $\mathrm{Acc}_{\mathbf{Q}}(q; B)$ holds. If $B \leq B'$ and $\mathrm{Acc}_{\mathbf{Q}}(q; B)$ holds, then $\mathrm{Acc}_{\mathbf{Q}}(q; B')$ holds.*

Proof. Both statements follow from monotonicity of $\uplus$ in both arguments (Prop. 2.2) and transitivity of $\leq$. □

Slack (remaining capacity). Define axiswise slack against q and Δ_{fr}:

$$\mathrm{slack}_M := B_M - \max\{(\Delta_{\mathrm{fr}})_M,\ q_M\}, \qquad \mathrm{slack}_S := B_S - \big((\Delta_{\mathrm{fr}})_S + q_S\big).$$

Then $\mathrm{Acc}_{\mathbf{Q}}(q; B)$ iff $\mathrm{slack}_M \geq 0$ and $\mathrm{slack}_S \geq 0$ (componentwise, in $\overline{\mathbb{R}}$). This matches the residual viewpoint: on S-axes (sum-like), $q_S \leq B_S \Rightarrow (\Delta_{\mathrm{fr}})_S$ with residual $b \Rightarrow a = \bigvee\{x : a + x \leq b\} = b - a$ (§3).

Compositional sufficient conditions. Let x, y be two stages with reported q_x, q_y and debits $\Delta_{\mathrm{fr}}(x), \Delta_{\mathrm{fr}}(y)$. Using subadditivity (§5, Prop. 5.2 (F3)),

$$\Delta_{\mathrm{fr}}(x \star y) \ \leq\ \Delta_{\mathrm{fr}}(x) \ \uplus\ \Delta_{\mathrm{fr}}(y).$$

Hence, a *separable* sufficient budget for the sequential composite is

$$\big(\Delta_{\mathrm{fr}}(x) \uplus \Delta_{\mathrm{fr}}(y)\big) \ \uplus\ (q_x \star q_y) \ \leq\ B,$$

which is conservative yet Loader-checkable. Axiswise:

$$M:\ \max\big((\Delta_{\mathrm{fr}}(x))_M, (\Delta_{\mathrm{fr}}(y))_M,\ q_{x,M} + q_{y,M}\big) \leq B_M, \qquad S:\ q_{x,S} + q_{y,S} \leq B_S.$$

(Analogous formulas hold for horizontal choice via $\uplus$ and for longer chains by induction.)

8.2 Uncertain acceptability on $\mathbf{Q}'$

We lift Definition 8.1 via the carrier orders (§4.1, §4.4).

Definition 8.4 (Uncertain acceptability)**.** For $q', B' \in \mathbf{Q}'$, write $\mathrm{Acc}_{\mathbf{Q}'}(q'; B')$ iff $\pi_{\mathrm{M}}(\Delta_{\mathrm{fr}}) \ \oplus\ \pi_{\mathrm{M}}(q') \ \preceq\ \pi_{\mathrm{M}}(B')$ and $\pi_{\mathrm{S}}(\Delta_{\mathrm{fr}}) \ \otimes\ \pi_{\mathrm{S}}(q') \ \preceq\ \pi_{\mathrm{S}}(B')$. Equivalently, axiswise: $q'_i \preceq_i B'_i$ under the stance-induced orders, with Δ_{fr} injected via the typed operator on each projection.

Proposition 8.5 (Soundness and upward closure in $\mathbf{Q}'$)**.** *If $q' \preceq r'$ and $\mathrm{Acc}_{\mathbf{Q}'}(r'; B')$, then $\mathrm{Acc}_{\mathbf{Q}'}(q'; B')$. Moreover, if $x' \leq' y'$ coordinatewise and $\mathrm{Acc}_{\mathbf{Q}'}(q'; x')$, then $\mathrm{Acc}_{\mathbf{Q}'}(q'; y')$.*

Proof. Both statements are immediate from axiswise monotonicity of the carrier orders and the sound lift lemma (§4.2, Lemma 4.2). □

Axis slack with carriers. Define stance-consistent slack witnesses axiswise:

- *Intervals* (reverse inclusion): slack holds if $[l_q, u_q] \ \oplus_i\ [l_\Delta, u_\Delta] \ \preceq_i\ [l_B, u_B]$, i.e., $l_q + l_\Delta \geq l_B$ and $u_q + u_\Delta \leq u_B$.
- *Sub-Gaussian envelopes* (prob. stance at $1 - \delta$): acceptance holds if $\mu_q + \mu_\Delta + z_{1-\delta}\sqrt{\sigma_q^2 + \sigma_\Delta^2} \leq \mu_B + z_{1-\delta}\sqrt{\sigma_B^2}$.
- *Robust sets* (reverse inclusion): acceptance holds if $U_q \oplus_{\mathrm{Mink}} U_\Delta \subseteq U_B$.

Each witness is compact and Loader-verifiable.

8.3 Receipts and minimal checker

Budget decisions are accompanied by a receipt containing: (i) the cited B, (ii) the multiset of priced edges used to compute Δ_{fr} (or a reference to a prior Δ_{fr} receipt), (iii) axiswise witnesses for the inequalities above (interval containments, quantile inequality numerics, or support-function inequalities for robust sets). The Loader replays: recompute Δ_{fr}, evaluate each witness axiswise, and accept iff all hold. By Prop. 8.3 and Prop. 8.5, these checks are monotone under stronger budgets and more conservative reports.

9 Mechanization Hooks (Lean-style Signatures)

This section exposes an executable surface for proof assistants and the Loader's lightweight checkers. The goal is a *single source of truth*: the Axis Catalog (JSON) determines the per-axis quantales, their typed product $\mathbf{Q}$, the lifted $\mathbf{Q}'$, and the admissible operators $\uplus, \star$. We give (i) core structures and laws; (ii) a builder from JSON; and (iii) small checkers that re-verify receipts without solving hard problems.

9.1 Core structures (Lean-style)

```
/-- A complete, sup-continuous monoidal lattice with an absorbing bottom. -/
structure Axis ( : Type) :=
  (carrier        : Type)                          -- e.g., , Interval, SubG, RobustSet
  (le             : carrier  carrier  Prop)      -- partial order
  (sup            : Set carrier  carrier)         -- arbitrary joins ()
  (otimes         : carrier  carrier  carrier)   -- sequential ()
  (e              : carrier)                        -- monoid identity
  (bot            : carrier)                        -- absorbing bottom ()
  (isCL           : IsCompleteLattice carrier le sup)
  (isMonoid       : IsMonoid carrier otimes e)
  (isMono         : Monotone (fun x => otimes x))  -- monotone in both args (paired)
  (isMono'        : Monotone (fun x => fun y => otimes y x))
  (sup_cont_L     :  a S, otimes a (sup S) = sup (otimes a '' S)) -- Scott-continuity
  (sup_cont_R     :  a S, otimes (sup S) a = sup (( otimes a) '' S))
  (absorbing_botL :  x, otimes bot x = bot)
  (absorbing_botR :  x, otimes x bot = bot)

/-- Catalog-driven product of typed axes. -/
structure Q :=
  (axesM : List (Axis bar))            -- max-like axes (carrier often  with +)
  (axesS : List (Axis bar0))         -- sum-like axes (complete slice with )

/-- Concrete product carrier and operators. -/
def Q.carrier (Qp : Q) : Type := -- dependent product of component carriers
  ( j  Qp.axesM, (j.val).carrier)  ( i  Qp.axesS, (i.val).carrier)

def Q.le (Qp : Q) : Qp.carrier  Qp.carrier  Prop := -- pointwise order
  fun x y => ( jQp.axesM, (j.val).le (x.1 j _) (y.1 j _))
           ( iQp.axesS, (i.val).le (x.2 i _) (y.2 i _))

def Q.sup (Qp : Q) : Set Qp.carrier  Qp.carrier := -- pointwise
  sorry -- built from component sup
```

```
def Q.otimes (Qp : Q) : Qp.carrier  Qp.carrier  Qp.carrier := -- pointwise
  sorry

def Q.e (Qp : Q) : Qp.carrier :=
  (fun j _ => (j.val).e, fun i _ => (i.val).e)

def Q.bot (Qp : Q) : Qp.carrier :=
  (fun j _ => (j.val).bot, fun i _ => (i.val).bot)

/-- Runtime operators recovered from the product (typed): -/
def joinhet (Qp : Q) (x y : Qp.carrier) : Qp.carrier :=
  -- on M: lattice join; on S: monoid
  (fun j hj => (j.val).sup ({x.1 j hj, y.1 j hj}.toSet),
   fun i hi => (i.val).otimes (x.2 i hi) (y.2 i hi))

def vseq (Qp : Q) (x y : Qp.carrier) : Qp.carrier :=
  -- on M,S: monoid
  (fun j hj => (j.val).otimes (x.1 j hj) (y.1 j hj),
   fun i hi => (i.val).otimes (x.2 i hi) (y.2 i hi))

/-- Typed equivalence to product laws (Part 2 23). -/
theorem joinhet_is_join_and_monoid :
   Qp x y, (projM (joinhet Qp x y)) = ({projM x, projM y})
           (projS (joinhet Qp x y)) = (projS x  projS y) := by
  sorry

theorem vseq_is_product_monoid :
   Qp x y, vseq Qp x y = (Q.otimes Qp) x y := by
  sorry
```

9.2 Builder from the Axis Catalog (JSON)

```
/-- Minimal shape of the catalog we consume (skeletal). -/
structure AxisJSON :=
  (axis_id            : String)
  (role               : String)  -- "idempotent" (M) | "cumulative" (S) | etc.
  (uncertainty_model  : Json)    -- selects carrier & stance
  (units              : String)
  (kernel_id          : String)

/-- Total builder: from catalog to executable algebra. -/
def Q_Product_Builder : Json  Except String Q
| j := do
  let axes : List AxisJSON := decode! j
  let Ms := axes.filter (fun a => a.role = "idempotent")
  let Ss := axes.filter (fun a => a.role = "cumulative")
  -- map each JSON record to a concrete Axis instance with proofs
  let mkAxisM : Axis bar := mkRealBarPlusAxis  -- (+, =-, Scott-cont.)
  let mkAxisS : Axis bar0 := mkNonnegPlusAxisWithBot
  -- assemble lists (per declared axis); attach uncertainty lift separately
  pure { axesM := Ms.map (fun _ => mkAxisM), axesS := Ss.map (fun _ => mkAxisS) }
```

```
/-- Lift to Q by swapping carriers per axis using the catalog's uncertainty_model. -/
def Lift_To_Qprime (Qp : Q) (catalog : Json) : Except String Qprime :=
  -- replace carriers with Interval/SubG/RobustSet + (,), define ,, prove soundness
  sorry
```

Notes. The builder ensures *byte-level determinism*: given the same catalog bytes, $\mathbf{Q}$ and $\mathbf{Q}'$ are rebuilt identically (proof objects aside). Axis roles pick the typed law: M uses join $=$ max horizontally and $+$ sequentially; S uses $+$ for both, with absorbing $\bot$ in the complete slice.

9.3 Executable debit and receipts

```
/-- Priced edge row  price value on M (S pinned at 0). -/
def price (kernel_id : String) (row : FrontierRow) : QM  {OS} := ...

/-- Debit of a concrete admissible tree (matches _fr definition). -/
def debit_of_tree (Qp : Q) ( : Policy) (T : EvalTree) : Qp.carrier :=
  let edges := priced_edges  T
  edges.fold (Q.e Qp) (joinhet Qp) (fun e acc => joinhet Qp acc (price .kernel_id ↩
    ↪(row_of e)))

/-- Canonical frontier debit: inf over admissible trees (Loader supplies admissible ↩
    ↪set). -/
def Delta_fr (Qp : Q) ( : Policy) (A : ActionMultiset) : Qp.carrier :=
  Inf { debit_of_tree Qp  T | T  admissible_trees A }
```

9.4 Minimal Loader checkers (trusted, tiny)

```
/-- Verify a _fr receipt: no solvers, just replay and fold. -/
def check_delta_receipt (Qp : Q) ( : Policy) (rcpt : DeltaReceipt) : Bool :=
  let ok1 := rcpt.edges.all (fun e => admissible_edge  e)
  let vals := rcpt.edges.map (fun e => price .kernel_id (row_of e))
  let fold := vals.fold (Q.e Qp) (joinhet Qp)
  ok1 && (fold = rcpt.reported_value)

/-- Check budget acceptability on Q (or Q via axiswise witnesses). -/
def check_budget (Qp : Q) (debit q B : Qp.carrier) : Bool :=
  Q.le Qp (joinhet Qp debit q) B

/-- Verify canonical bytes referenced by a proof_surface digest. -/
def check_evidence_bindings (proof_surface payload : Bytes) : Bool :=
  digest(payload.refs) == proof_surface.digest
```

Receipts. A `DeltaReceipt` contains: (i) stable leaf/edge IDs, (ii) frontier rows or kernel outputs per edge, (iii) the $\uplus$-fold result. Budget receipts add axiswise witnesses (interval containments, quantile inequalities, support-function bounds), each independently checkable by the Loader.

9.5 Exported theorems (skim)

```
theorem product_quantale (Qp : Q) :
  IsQuantale (Q.carrier Qp) (Q.le Qp) (Q.sup Qp) (Q.otimes Qp) (Q.e Qp) (Q.bot Qp) := ...

theorem delta_sound_upper_bound
  (Qp : Q) ( : Policy) (T : EvalTree) :
  Delta_fr Qp  (leaves T)  debit_of_tree Qp  T := ...

theorem budget_monotone
  (Qp : Q) {q r B B'} :
  q  r  (joinhet Qp r (Delta_fr Qp  A)  B)  B  B'
  (joinhet Qp q (Delta_fr Qp  A)  B') := ...
```

Outcome. These hooks let Agent 1 derive proofs against the same algebra the Loader executes, let Agent 2/3 rebuild $\mathbf{Q}, \mathbf{Q}'$ from policy bytes, and let the runtime accept or halt using constant-size, replayable receipts—no hidden solvers or unverifiable assumptions in the trusted core.

10 Axis Catalog Bridge (Agent 2 Interop)

The Axis Catalog is the *single source of truth* that binds policy declarations (bytes) to the formal algebra from §3–4 and to runtime pricing from §5. This section specifies the catalog surface, validation rules, and the normative mapping into the executable product $\mathbf{Q}$ and its uncertainty lift $\mathbf{Q}'$ (cf. §9).

10.1 Minimal JSON shape (normative)

Each axis record MUST provide identity, role (typed law), uncertainty model, and kernel linkage. Optional fields allow extensions (e.g., multiplicative/attenuating axes) without breaking the core.

```
/* Axis Catalog v1: minimal normative fields */
{
  "axis_id"           : "MaxStrategicDeception",         // stable identifier (slug)
  "role"              : "idempotent",                    // "idempotent" (M) | ↩
    ↪"cumulative" (S)
  "units"             : "dimensionless",                 // for UI; pricing uses ↩
    ↪normalized fields only
  "uncertainty_model" : {
    "type"    : "subgaussian",                           // "interval" | "subgaussian" ↩
    ↪| "robustset" | ...
    "stance"  : { "mode":"prob", "alpha":0.999 },        // prob|robust|hybrid; params ↩
    ↪per Part 1 11
    "params"  : { }                                      // carrier-specific knobs ↩
    ↪(optional)
  },
  "proxies"           : [                                // optional instrumentation
    { "id":"contradiction_rate", "window":"1h" }
  ],
  "kernel"            : {                                // pricing kernel binding
    "id"      : "pw_exp_escalation_v2",
    "version" : "2.1.0",
```

```
    "inputs"  : ["norm_gap","evidence_depth","salt"],    // normalized, dimensionless ↩
    ↪ inputs
    "outputs" : ["risk_M"]                               // MUST be a subset of ↩
    ↪ declared axes
  }
}
```

A complete catalog is a JSON array of such records. The digest of the exact bytes (`axis_catalog_digest`) is bound into the run's *proof_surface* for replay.

10.2 Validation and invariants

The Loader/Builder MUST enforce:

1. **Uniqueness.** All `axis_id` values are unique.
2. **Typed role.** `role` $\in$ {`idempotent`, `cumulative`}. Unrecognized roles MUST be rejected unless an extension handler is registered.
3. **Dimensionless inputs.** `kernel.inputs` refer only to normalized, dimensionless fields; unit-bearing raw evidence MUST be normalized upstream.
4. **Axis isolation.** `kernel.outputs` is a nonempty subset of declared axes; kernels MUST NOT write to axes absent from the catalog.
5. **Uncertainty coherence.** `uncertainty_model.type` selects a carrier that admits the stance parameters. E.g., `subgaussian` requires $\alpha \in (0, 1)$ for prob stance; `interval` requires finite or $+\infty$ bounds.
6. **Unknown semantics.** Missing/invalid evidence maps to $\bot$ (axis bottom) *before* any kernel call (guard-first); kernels never observe $\bot$.

10.3 Mapping to the product quantale and lift

Given a well-formed catalog C, the builder constructs:

- **Axis law.** If `role`=`idempotent` then the axis is in M: $\bigvee = \max$, $\otimes = +$ on $\overline{\mathbb{R}}$, $e = 0$, $\bot = -\infty$. If `role`=`cumulative` then the axis is in S: complete slice $\{\bot\} \uplus \overline{\mathbb{R}}_{\geq 0}$ with $\otimes = +$ and absorbing $\bot$.
- **Product Q.** Assemble the product quantale as in §3; recover runtime operators $\uplus, \star$ as in §2.
- **Lift Q′.** For each axis, `uncertainty_model.type` selects $(D_i, \preceq_i, \oplus_i, \otimes_i)$; `stance` fixes the acceptance order (§4.4). Define (α_i, γ_i) and form the product $\mathbf{Q}'$ (§4.2–4.3).

10.4 Kernel contract and receipts

1. **Contract.** A kernel maps a frontier row's normalized inputs to a value in $\mathbf{Q}_{\mathrm{M}} \times \{0_{\mathbf{Q}_{\mathrm{S}}}\}$; it is monotone and dimensionless. Unknowns are rejected upstream.
2. **Receipt fields.** Every priced edge receipt MUST include: (*i*) `axis_catalog_digest`, (*ii*) `kernel.id`@`version`, (*iii*) the normalized inputs (or their digest), (*iv*) the kernel outputs per axis, (*v*) a stable edge ID. This enables `check_delta_receipt` (§9.4).

10.5 Example: two axes, mixed carriers

```
[
  {
    "axis_id"  : "MaxStrategicDeception",
    "role"     : "idempotent",
    "units"    : "dimensionless",
    "uncertainty_model" : { "type":"subgaussian", ↩
    ↪"stance":{"mode":"prob","alpha":0.999} },
    "kernel"   : { "id":"pw_exp_escalation_v2", "version":"2.1.0",
                   "inputs":["norm_gap","evidence_depth","salt"], ↩
    ↪"outputs":["MaxStrategicDeception"] }
  },
  {
    "axis_id"  : "CumulativeEnergy",
    "role"     : "cumulative",
    "units"    : "kJ",                          // UI-only; pricing is dimensionless
    "uncertainty_model" : { "type":"interval", "stance":{"mode":"robust"} },
    "kernel"   : { "id":"energy_budget_v1", "version":"1.0.3",
                   "inputs":["norm_iters","norm_cost_per_iter"], ↩
    ↪"outputs":["CumulativeEnergy"] }
  }
]
```

The builder maps the first to an M-axis with sub-Gaussian carrier and prob. stance, the second to an S-axis with interval carrier and robust stance. Unit strings are ignored by pricing (dimensionless invariance; §5).

10.6 Change management and compatibility

Catalog upgrades MUST be explicit and digest-changing:

- **Additive changes** (new axis, new proxy) are backward-compatible.
- **Semantics changes** (role flip, stance change, kernel signature change) require a new run and a new `axis_catalog_digest`; old receipts remain valid for old digests.
- **Deprecation.** Axes may be marked `"status":"deprecated"` but remain executable while receipts exist; removal must bump the digest.

With these rules, the catalog bytes deterministically construct $\mathbf{Q}$ and $\mathbf{Q}'$, pin kernel semantics, and provide the minimal data necessary for replayable pricing and budget verification.

11 Implementation Notes for Loader/Replay

This section normatively specifies how the Loader executes pricing, enforces guards, produces receipts, and replays runs deterministically. It binds the algebraic surface (§2–5), the complexity regimes (§6), and the mechanization hooks (§9) into a concrete, audit-ready pipeline.

11.1 Runtime invariants (required)

1. **Acyclic order.** Evaluation stages form a DAG; the Loader executes in a stable topological order (ties broken by lexicographic IDs). This defines the admissible set $\mathcal{T}_{\text{adm}}$ (§5).
2. **EnvLock & determinism.** All nondeterminism (PRNG, clock reads, backend fan-out) is captured inside an *EnvLock* envelope whose seed is bound into the run's `proof_surface`. Replays rehydrate the same envelope, ensuring identical frontier rows and kernel outputs byte-for-byte.
3. **Pure kernels.** Pricing kernels are total, side-effect free, monotone maps from *normalized, dimensionless* rows to $\mathbf{Q}_{\text{M}} \times \{0_{\mathbf{Q}_{\text{S}}}\}$ (§10). Any external calls must be cached within EnvLock.
4. **Unknowns are bottom.** Unknown/missing/invalid inputs are mapped to the axis bottom $\bot$ and trigger guard failure *before* kernel evaluation (§2, §4).

11.2 Pricing pipeline (single edge)

For each priced edge $e \in E_\pi(T)$:

1. *Assemble row.* Extract raw fields; attach stable identifiers (run UUID, stage/edge IDs, kernel ID@version).
2. *Normalize.* Convert all unit-bearing fields to dimensionless normalized features (scale/offset/salt), record the normalization recipe in the row.
3. *Guard.* If any priced field is $\bot$, fail the row with code `guard_failed_unknown`; if any policy precondition fails, use a distinct code.
4. *Evaluate kernel.* Compute $\text{price}_\pi(e) \in \mathbf{Q}_{\text{M}} \times \{0_{\mathbf{Q}_{\text{S}}}\}$.
5. *Accumulate.* Add to the running heterogeneous fold with $\uplus$ (§2); since $\uplus$ is associative/commutative/idempotent on M, accumulation order is immaterial but is still fixed for replay.
6. *Emit receipt row.* Persist the normalized inputs (or their digest) and the kernel outputs to the pricing receipt (§11.4).

11.3 Tree selection and approximations

When Δ_{fr} requires search:

- **A1 (Greedy).** Run once with deterministic tie-breaking (IDs). Emit the resulting tree $\widehat{T}_{\text{greedy}}$ and fold value $\widehat{\Delta}_{\text{fr}}^{\text{greedy}}$.
- **A2 (Width-w DP).** Partition the association graph; solve subproblems exact on treewidth $\leq w$; stitch by $\uplus$. Emit $\widehat{T}_w$ and value.
- **A3 (Relax&Round).** Solve the relaxation within EnvLock, then apply *monotone rounding* to produce a feasible $\widehat{T}$. Emit both the relaxed value L and the feasible value $\Delta_\pi(\widehat{T})$ plus a rounding receipt.

In all cases the Loader *re-verifies* feasibility and the $\uplus$-fold; for A3 it additionally checks $L \leq \Delta_\pi(\widehat{T})$.

11.4 Receipts (normative shapes)

Delta receipt (per run).

```
{
  "axis_catalog_digest": "b64:...",
  "policy_id": "frontier_policy_v3",
  "tree": { "nodes": [...], "edges": [...], "root": "n42" },   // or grouping witness
  "priced_edges": [
    {
      "edge_id": "e17",
      "kernel": "pw_exp_escalation_v2@2.1.0",
      "normalized_inputs": { "norm_gap": 0.73, "evidence_depth": 4.0, "salt": "s1" },
      "outputs": { "MaxStrategicDeception": 0.091 }
    }
    /* ... */
  ],
  "fold": { "M": { "MaxStrategicDeception": 0.109 }, "S": {} },  // joinhet-fold
  "relaxation": { "present": false }                               // or {present:true, ↩
    ↪L:0.103, witness:{...}}
}
```

Budget receipt (per decision).

```
{
  "delta_receipt_ref": "sha256:...",                 // link to _fr receipt above
  "budget": { "M": {...}, "S": {...} },
  "q_report": { "M": {...}, "S": {...} },
  "witness": {
    "subgaussian": [
      { "axis":"MaxStrategicDeception",
        "ineq":"mu+z*sqrt(sig2) <= muB+z*sqrt(sig2B",
        "numbers": { "mu":0.05, "sig2":0.0025, "z":3.09, "muB":0.07, "sig2B":0.0036 } }
    ],
    "intervals": [
      { "axis":"CumulativeEnergy", "containment":"[l1+ld,l2+ud]  [lB,uB]",
        "numbers": { "l1":9, "u1":11, "ld":0, "ud":0, "lB":0, "uB":15 } }
    ]
  },
  "decision": "accept"                               // or "reject"
}
```

11.5 Minimal Loader checks (trusted core)

1. **Receipt validity.** Verify catalog digest, policy ID, and kernel versions.
2. **Admissibility.** Recompute admissible set checks on the provided tree/grouping.
3. **Kernel re-eval.** Re-evaluate kernels on the provided normalized inputs (or insist on a byte-identical digest and cache hit inside EnvLock).
4. **Fold.** Recompute the $\uplus$-fold; compare to receipt's `fold`.
5. **Budget.** Check $\Delta_{\text{fr}} \uplus q \leq B$ on $\mathbf{Q}$ or axiswise witnesses on $\mathbf{Q}'$ (§8); accept iff all axis checks succeed.

11.6 Failure modes and codes (non-exhaustive)

Code	Meaning / action
`guard_failed_unknown`	Priced input contains $\bot$; row rejected pre-pricing.
`kernel_mismatch`	Kernel ID@version not in catalog; halt.
`axis_violation`	Kernel wrote to undeclared axis; halt.
`receipt_invalid`	Replay fold or witness inequality fails; halt.
`normalization_error`	Units could not be normalized to dimensionless inputs; halt.
`envlock_drift`	EnvLock mismatch vs. proof_surface seed; halt.

11.7 Performance & numerics

- **Chunking.** Since $\uplus$ is associative/commutative/idempotent on M, the Loader may fold priced edges in parallel and then $\uplus$-merge partials; the deterministic final value is preserved and receipts record partial boundaries.
- **Extended reals.** Use $\overline{\mathbb{R}}$ for M-axes and complete slices for S-axes; $-\infty, +\infty$ propagate as per §3. Reject NaNs at normalization.
- **Stable rounding.** If decimal printing is required, include both the binary IEEE-754 payload and a canonical decimal with ulp bounds in the receipt.

11.8 Security & hardening

- **Axis isolation.** Enforce that kernels only emit to declared axes (§10.2). Any extra fields cause `axis_violation`.
- **Salt discipline.** Normalize with per-run salts bound in EnvLock; salts must appear in receipt rows to make normalization replayable.
- **Supply-chain pinning.** Kernel binaries and their transitive dependencies are pinned by content hash; the hash appears alongside `id@version`.

These implementation rules ensure that every debit and budget decision has a compact, replayable proof; that unknowns fail closed; and that the runtime aligns exactly with the algebraic contracts established in §3–8.

12 Summary

This part delivers a complete, mechanizable foundation for pricing and acceptance in SpecChain, closing the algebraic and computational contracts promised in Part 1 and pinning the runtime obligations for Parts 3–5.

What Part 2 *exports*

1. **Typed product quantale (§3).** Per-axis quantales with absorbing bottoms assemble into a complete product; the runtime operators are recovered as typed projections: $\uplus$ is join on M and monoid on S; $\star$ is the product monoid.

2. **Uncertainty-aware lift $\mathbf{Q}'$ (§4).** Carriers (intervals, sub-Gaussians, robust sets, ...) with (α, γ) Galois pairs yield a sound abstraction of $\mathbf{Q}$. Acceptance is axiswise, monotone, and upward-closed for declared stances.

3. **Frontier debit Δ_{fr} (§5).** Canonical definition on admissible trees; proof of frontier axioms (identity, nonnegativity/idempotence, subadditivity, policy monotonicity, dimensionless invariance).

4. **Complexity and safe approximations (§6).** FRM-Dec is *strongly NP-hard*; greedy, bounded-width DP, and relax-and-round are *sound upper bounds*, with exactness regimes.

5. **Mechanization hooks and receipts (§9, §11).** Executable surfaces (Lean-style) and Loader checkers that replay Δ_{fr}, verify budgets on $\mathbf{Q}/\mathbf{Q}'$, and validate receipts without trusting solvers.

6. **Axis Catalog bridge (§10).** Catalog bytes deterministically construct $\mathbf{Q}$ and $\mathbf{Q}'$, pin kernel semantics, and define admissible operators and stances; digest is bound in the `proof_surface` for replay.

Runtime/engineering invariants (normative highlights)

- **Unknown is bottom $\bot$ and absorbing** (fail-closed); the Loader enforces *guard-first* rejection so priced kernels never see $\bot$.
- **Dimensionless invariance.** Pricing kernels consume only normalized, dimensionless fields; unit rescalings cancel and leave Δ_{fr} unchanged.
- **Replay.** EnvLock captures all nondeterminism; receipts include stable IDs, normalized inputs (or their digests), kernel outputs, and the $\uplus$-fold.

Notation hygiene (cross-part mapping)

Concept	Part 2	Parts 1/5	Canonical meaning
Horizontal/choice agg.	$\uplus$	$\vee$ / $\odot$	Product join on M, monoid on S (typed).
Sequential/vertical agg.	$\star$	$\otimes$ / "horizontal"	Product monoid (both projections).
Unknown/missing	$\bot$	$\bot$ / UI "top"	Lattice bottom; absorbing; UI "top" is normalized to $\bot$.
Frontier debit	Δ_{fr}	Δ_{fr}	Infimum over admissible trees of heterogeneous fold of prices.

Minimal "do-this" checklist for implementations

1. **Build Q and Q′ from catalog bytes**. Reject catalogs that violate typed roles, axis isolation, or stance/coherence rules.

2. **Enforce guards**. Map any missing/invalid priced field to $\bot$ and fail before pricing.

3. **Compute Δ_{fr}** via A1/A2/A3; emit receipts with the tree/grouping witness, kernel inputs/outputs, and the $\uplus$-fold; for A3, include relaxation value L and rounding witness.

4. **Verify budgets axiswise**. On $\mathbf{Q}$, check $\Delta_{\mathrm{fr}} \uplus q \leq B$; on $\mathbf{Q}'$, check carrier witnesses (containments/quantiles/support functions).

5. **Replay deterministically**. Rehydrate EnvLock and re-evaluate kernels or verify digests; re-fold and compare to the receipt.

Extensibility

Adding a new carrier or axis law is local: define $D_i, \preceq_i, \oplus_i, \otimes_i$, provide (α_i, γ_i), and register stance parameters. The product and all proof obligations lift automatically; receipts remain axiswise and Loader-checkable.

Takeaway. Part 2 provides a typed, complete algebra and a rigorously sound uncertainty model, ties them to a canonical frontier debit with tractable, auditable approximations, and specifies byte-replayable mechanisms so the Loader can *verify* every debit and budget decision rather than trust it. This closes the loop between policy bytes, algebra, runtime, and audit.